TOHSENO

An Open-Source Factory for Independent iOS Apps

GENESIS WHITEPAPER

Version 1.0 --- July 27, 2026

Status: Foundational protocol specification

Origin: JP Franetovic / Anky, Inc.

Give every idea a shot.

Let reality continue it.

Take another one.

ONE COHERENT INTENTION / ONE INDEPENDENT APPLICATION / ONE ENCOUNTER WITH REALITY

"Real artists ship."

STEVE JOBS, 1983

This document exists in service of one intent: that any person who carries an idea for software should be able to give it a body — to build it, hold it, run it on the phone in their pocket, and learn from it what it wants to become. TOHSENO is that intent given a system. It is written deliberately in the language of the Apple ecosystem, and in the spirit of the man who spent his life insisting that the computer belongs in the hands of ordinary people — as a bicycle for the mind.

"Everything around you that you call life was made up by people that were no smarter than you. And you can change it, you can influence it, you can build your own things that other people can use. Once you learn that, you'll never be the same again."

STEVE JOBS, 1994

Abstract

Software is becoming easier to write than it is to carry.

Artificial intelligence has collapsed the cost of producing code. A person can describe an application in ordinary language and receive substantial working software in minutes. But generated code is not yet a coherent creative practice, and a rendered screen is not yet an application a person truly owns. The unresolved problem is the passage from intention to reality: where does the application live, can the person run it on the device where it matters, can it survive the model and company that produced its first form, and what happens after the person touches it and discovers the idea in their mind was incomplete?

TOHSENO proposes a new unit of software production called a **Shot**.

Taking a Shot is the act of giving one coherent intention a bounded, executable attempt. A **Shot** is the stable application lineage created by that act: an independently owned native iOS application that can be used, verified, evolved, published, or left at rest without depending on TOHSENO for its continued existence.

A Shot is a question asked of reality in executable form. Its first output is an application. Its second output is **evidence**: whether the application works, whether the builder uses it, what feels wrong, what feels unexpectedly alive, and what the next intention should become. Across many Shots this evidence accumulates into a body of work, and the body of work develops the builder's taste. The application may rest; what the builder learned remains.

TOHSENO is tooling for the Apple ecosystem, on purpose and permanently. The Mac is the local factory. The iPhone is the site of truth. The App Store is the summit of distribution. A person begins with an intention and ends with an ordinary Xcode-buildable repository they can carry forward without TOHSENO's permission. No TOHSENO account, network, token, or server is required to create, own, use, or continue a Shot.

The objective is not to make every idea successful.

The objective is to make reality cheap enough to answer.

Contents

1 Status and Scope

This paper defines the enduring object and creative method at the center of TOHSENO. It is a constitutional document, not an implementation manual. Exact schemas, command names, file layouts, and verification procedures belong to separate, versioned specifications that may change as models, languages, and tools change. What is written here is what must remain true even as all of those change.

This paper defines: the Shot; the act of taking one; coherence; Contact; Evidence; Evolution; the contact sheet; the boundary between human judgment and artificial intelligence; the conditions of ownership and continuity; the properties of a conforming implementation; and the constitutional limits on future network, economic, and publication layers.

2 The Observation

2.1 Software had to become a project before it could become an experience

A person can carry a coherent piece of software in mind long before they know how to build it. The idea is often personal: an application that supports a family ritual, a game for a narrow community, a tool shaped around one person's work, a small behavior that no existing product understands.

Historically, the idea had to become a *project* before the person could experience it — technical knowledge, a framework, infrastructure, accounts, money, and enough confidence to justify all of those costs. The idea had to win an argument before it was allowed to acquire a body. People learned to ask: *Is this worth six months? Could this become a business? What if I begin and fail?* These questions were reasonable responses to expensive production. They were not good conditions for discovery.

Expensive beginnings also distort the creator. When every application is a large bet, every idea is pressured to become *the* precious bet: the builder plans before using, defends before learning, and grows attached precisely because reality has not yet been allowed to contradict the idea. Starting feels dangerous, stopping feels shameful, and the cost already paid becomes the argument for continuing.

2.2 Cheap generation is not yet a creative practice

Coding agents change the economics of first implementation. Natural language is becoming a legitimate front door to computation. This is an extraordinary opening — but cheaper code does not automatically produce better creators. Without a coherent method, lower production costs create an endless stream of fragments: demos, abandoned prompts, and software that was generated but never truly encountered.

A pile of untouched prototypes is not a body of work.
It is deferred reality.

The relevant distinction is not between people who generate many applications and people who generate few. It is between generation and contact. Generation produces an object. Contact produces knowledge.

2.3 The new bottleneck

As implementation becomes cheap, the scarce capacity moves. It is no longer the ability to translate instructions into syntax. It is the ability to recognize a coherent intention, give it a bounded first form, carry that form into reality, notice what use reveals, form the next intention from evidence rather than fantasy, and develop judgment across a body of work.

This is the emerging craft. Models can compress execution. They cannot live the builder's life, touch the application with the builder's hands, or decide what the experience means. Models help make the object. Reality and the builder continue the work.

3 Shoulders of Giants

TOHSENO did not invent its parts. It inherits deliberately, and it should be understood through its inheritance. The shortest true description is:

TOHSENO is Vite for iOS apps —
if Vite also insisted you use what you build.

3.1 What is recreated

From Vite and the modern web toolchain: the conviction that the distance between writing and seeing must approach zero, and that a great default pipeline — scaffold, run, hot loop — is itself a creative instrument. TOHSENO recreates this for the Swift toolchain: intention to running Simulator with as little ceremony as the platform allows.

From Rails: convention over configuration. The builder should not choose an architecture before they have experienced their own application. TOHSENO composes an opinionated baseline and lets infrastructure earn its existence through application necessity.

From create-react-app: the eject. CRA proved that a scaffold must offer a full exit — and taught, by its decline, that ejection bolted on as an escape hatch is not enough. TOHSENO promotes ejection from feature to invariant: every Shot is ejectable *from birth*, because it is never anything other than an ordinary Xcode-buildable repository in the first place.

From photography: the contact sheet. Many exposures placed side by side so the photographer

can see not only individual images but patterns in their way of looking. Most frames miss. The roll continues. The eye improves.

3.2 What is evolved

From local-first software (Ink & Switch and the researchers around it): the principle that the artifact on your machine is the real one, and that no service may be its hidden landlord. TOHSENO extends local-first from documents to entire applications, and from data ownership to *production* ownership: the factory itself runs on the builder's Mac.

From the Lean Startup and the prototyping tradition (build – measure – learn; sketching as thinking): the loop of bounded attempt and honest observation. TOHSENO evolves it in two ways: the loop is personal before it is commercial — the first market is the builder's own life — and the observation is split into two kinds of evidence, machine and lived, that must never be confused.

3.3 What is net new

Three things in this paper are, to our knowledge, original as protocol commitments rather than as good intentions:

1. **The Shot** as the unit of ownership: a stable application lineage, not a prompt, session, build, or deploy, created by a bounded act and continued by evidence.
2. **The evidence discipline**: Machine Evidence and Lived Evidence as distinct, bounded categories, with every readiness claim limited to the evidence that actually exists.
3. **The authority boundary**: artificial intelligence may propose; deterministic machinery may prove; the builder may authorize; reality may answer. A coding agent cannot certify its own work.

4 The Shot

4.1 The name

TOHSENO is **ONESHOT** reversed. "One shot" names the act: one coherent intention receives a real attempt at becoming software. The reversal names the method: TOHSENO does not begin with infrastructure and ask what product can justify it. It begins with the human intention and grows only the infrastructure the application proves it needs. The first question is not *which framework, backend, or deployment architecture?* The first question is: *what should exist?*

4.2 Taking a Shot

Taking a Shot is the act of giving one coherent intention a bounded, executable attempt. It is more than asking a model to generate code: it means allowing the intention to acquire consequences. While an idea remains in the mind it can remain perfect; reality cannot resist it. Taking a Shot forfeits that protection. The application may confirm the idea, reveal a deeper one, feel wrong immediately, become useful in an unpredicted way, or prove that nothing more should be built. Each of these is a legitimate outcome of creation.

4.3 Definition

A Shot is a coherent software intention embodied as a stable, independently owned native iOS application lineage.

The Shot begins when its first independently operable form is created and continues through meaningful Evolutions. It is embodied as an ordinary native iOS repository containing its source, history, manifest, operating knowledge, verification machinery, and evidence. The repository is the current body of the Shot; a particular build is one state of that body; the Shot is the continuing application-level object.

A useful way to understand a Shot is as a question asked of reality in executable form. An application that makes a person write before opening social media asks: *what changes when writing becomes the threshold to consumption?* The first implementation does not answer the question. It creates the instrument through which an answer can begin to emerge.

4.4 Coherence

"One intention" does not mean one sentence, one feature, or one screen. An intention may include prose, sketches, references, data examples, and several related capabilities. It is *coherent* when the material points toward one application-level object whose first form can be understood as a whole. A coherent Shot can answer four questions:

1. What does this application do? (One durable sentence.)
2. For whom, or in what context, does it first exist?
3. What complete experience must its first form make possible? (One complete loop, not a collection of screens.)
4. What is intentionally outside that first form?

Coherence is not completeness. A Shot may be ambitious in implication while bounded in first form. The first form should be small enough to arrive and real enough to answer. When one intention contains several unrelated products, the builder should narrow it or take several Shots.

4.5 Identity, ownership, and continuity

A Shot receives an opaque, stable identifier at creation, recorded in its manifest and carried through every Evolution. The identifier is machine-checkable. It is deliberately not derived from properties that may legitimately change: name, code, visual design, repository path, version, model provider, or distribution state.

The identifier preserves continuity; the *boundary* of continuity remains a human judgment, guided by one rule: a later change is an Evolution of the same Shot when it is best understood as an answer to something learned from the existing application. When the center of the inquiry changes enough that the new application is asking a different question, a new Shot should be taken.

Ownership is material. A Shot has exactly one Builder of record at a time, and ownership is possession of the repository: whoever holds the repository, its history, and its manifest holds the Shot. Transfer of the Shot is transfer of the repository. A copy made without the intention of continuing the same inquiry is a fork — ordinary software with its own future — and collaboration within one Shot is governed by ordinary software collaboration under terms the Builder of record sets. The protocol adds no ownership machinery beyond this, because any machinery it added would become the landlord the protocol exists to abolish.

4.6 What a Shot is not

A Shot is not a prompt, a chat session, a model response, a screenshot, a temporary preview, a Git commit, an Xcode build, a template, a ticket, a token, an account, a workspace of unrelated applications, or a remote object that ceases to function when the generating service disappears. These things may contribute to a Shot, describe it, finance it, verify it, or represent one state of it. They are not the primitive.

5 Contact

5.1 Definition

Contact occurs when a person encounters a functioning form of the Shot in the context where its central behavior can become true or false. In practice this means touching and using the application in Simulator or on an iPhone; the strongest first Contact occurs on the device, in the situation, for which the application was intended.

A successful build is not yet Contact. A screenshot is not Contact. Reading generated source is not Contact. Contact requires experience: the person moves through the application, their thumbs meet the interface, their routine accepts the tool or rejects it, and the original idea loses its immunity from reality.

After Contact, the protocol asks one canonical question:

What did using this version teach you
about what this application wants to become?

The wording is deliberately different from *what features should be added?* The answer may be that a capability should disappear, that the application should become smaller, that the real need is adjacent to the original idea, or that nothing needs to happen yet. The next intention should emerge from what was learned, not from the mere availability of more generation.

5.2 Two kinds of evidence

Every Shot produces two outputs: the application, and evidence. Evidence has two forms, and they must not be confused.

Machine Evidence establishes bounded technical facts: a plan validated, a repository created, a hash matched, a test passed, a build succeeded, a Simulator launched, an application ran on a device.

Lived Evidence records what use revealed: the central gesture felt natural or forced; the application was opened again without prompting; the intended routine did not survive contact; one screen contained the actual product; another person understood the application differently; the builder discovered a sharper need.

A compiler cannot prove usefulness. A feeling cannot prove security. A passing test cannot prove demand. A truthful protocol states the strength of each form of evidence and no more.

5.3 The contact loop

```
coherent intention -> bounded first form -> Contact
       ^                                        |
       |                              machine + lived evidence
       |                                        |
```

```
next intention  <----------------------------+
     |
Evolution, rest, publication, or another Shot
```

The first intention creates the Shot. Reality continues it. Each pass through the loop makes the application more grounded and the builder's judgment more specific. The protocol compresses the distance between these events; it does not remove the events themselves.

5.4 Evolution

An Evolution is a meaningful change to the same Shot made in response to a later intention. Not every commit is an Evolution: formatting, dependency updates, and incidental fixes may occur inside one. An Evolution marks a semantic step the builder recognizes as an answer to something learned, and should be able to state what was observed, what changed in the builder's understanding, what the next intention became, what machine evidence exists, and what remains unknown. This record exists to preserve continuity, not to expose private thought.

5.5 Rest, and the danger of abundance

A Shot may rest. Rest is not failure, deletion, or death; it is the absence of a current intention to continue. A resting Shot remains whole — it can be opened, used, published, or returned to. The protocol must not turn ordinary creative silence into administrative guilt: there is no obligation to maintain every application forever, write a postmortem, or publicly declare that an idea failed.

The same cheapness that permits rest creates a danger: a person may repeatedly generate first forms without inhabiting any of them, using novelty to avoid judgment. TOHSENO therefore must not optimize for the number of applications generated. It must optimize for honest creative loops completed. The default action after a first form is not *generate another application*. It is *use this one*. Without contact, abundance becomes debris. With contact, abundance becomes practice.

5.6 Most Shots miss

Most Shots will not become enduring public products. Some will not survive first Contact; some will be used for a week; some will reveal that the original problem was not real; some will become private tools; some will become companies, public goods, or cultural objects. A Shot can miss as a market and still land as a measurement. It can fail to deserve another Evolution and still make the next intention more truthful. The possibility of missing is not a defect. It is why the cost of taking another Shot must remain low.

6 The Contact Sheet

Together, a builder's Shots form a **contact sheet**: not merely a directory, but the visible body of work. Applications that would otherwise disappear into temporary chats and forgotten folders become concrete objects that can be used, compared, continued, or left at rest. The contact sheet changes the question from *which one idea deserves my identity?* to *what am I learning through the things I make?*

Across the sheet, patterns emerge. The builder discovers they repeatedly build tools around thresholds, memory, play, or attention; that their applications come alive when complexity is removed; which instincts are imitative and which are distinctive. The body of work is not a side effect of the protocol — it is one of its central outputs.

This is where taste comes from. Taste is not preference declared in advance; it is judgment compressed from repeated contact — *this interaction feels heavy; this one disappears into the hand; this screen is impressive but unnecessary; this idea deserves another Evolution; this one can rest*. A model can suggest these distinctions and a market can reward some of them, but the builder must still encounter and recognize them. The contact sheet gives that recognition somewhere to accumulate.

A person becomes a software creator not because a platform grants the title, but because they repeatedly complete the loop between imagination and reality. Natural language is a new front door, not the whole house: some Shots will eventually require deep engineering, security work, or legal review, and the protocol does not pretend otherwise. But a person should not need all of that before they can experience the first coherent form. They enter through intention, learn through ownership, and deepen their technical understanding as the consequences of the Shot require it.

7 The Protocol

TOHSENO is both a creative protocol and a technical protocol, and each protects the other. Without the creative half, TOHSENO would be another code-generation system, indifferent to whether the builder ever encounters what it produces. Without the technical half, the philosophy would depend on whichever provider currently hosts the experience — the builder encouraged to create while unable to carry the creation away.

7.1 The creative protocol

1. Begin with one coherent intention.
2. Bound its first form.
3. Give it an executable body.
4. Bring that body into Contact.

5. Distinguish machine evidence from lived evidence.
6. Form the next intention from what was learned.
7. Evolve the same Shot, let it rest, publish it, or take another.
8. Preserve the body of work.

7.2 The technical protocol

The technical protocol ensures the artifact is independently owned, locally inspectable, operable without any TOHSENO service, identified across Evolutions, private by default where raw intention is concerned, verifiable through executable evidence, explicit about external dependencies, and able to outlive the model and factory that produced it. It turns creative agency into material agency.

7.3 Actors

A conforming system distinguishes six actors.

The Builder — the human who supplies the intention, owns the artifact, experiences the application, interprets lived evidence, authorizes external actions, and decides what happens next. The Builder need not begin as a programmer. The Builder remains the source of intention and judgment.

The Factory — the deterministic system that protects the creation process: it normalizes private input, validates bounded plans, establishes stable identity, composes the baseline, records provenance, invokes coding providers, verifies results, and presents evidence. The Factory carries protocol ceremony so the Builder does not have to. It is infrastructure for agency, not the owner of the outcome.

The Intelligence Provider — the selected model, agent, or coding environment that interprets meaning and performs open-ended implementation work. Powerful, and replaceable. It is not the Shot's identity, and it does not own authority merely because it generated the code.

The Platform Toolchain — Swift, Xcode, Simulator, iOS, and Apple's signing and distribution systems: the machinery that establishes platform-specific facts.

Reality — the context in which the application is used: the builder's own behavior, the device, time, other people, actual data, and consequences the initial instruction could not predict. Reality does not produce a machine-readable answer. It resists, confirms, surprises, and reveals.

The Shot — the independently owned application lineage around which every other actor is temporary. The Factory may help create it, a model may help implement it, a platform may compile it, a network may later record it. None of these becomes the Shot.

7.4 The authority boundary

Artificial intelligence may propose.
Deterministic machinery may prove.
The Builder may authorize.
Reality may answer.

Artificial intelligence may interpret an intention, propose a plan, design an interface, write source, create tests, and suggest Evolutions. It may not silently grant itself authority over Shot identity, private provenance, protected composition rules, verification outcomes, credentials, spending, signing, publication, deployment, App Store submission, token creation, or claims of success.

A coding agent cannot certify its own work. An agent saying "the application is ready" is a progress signal, not proof. A conforming Factory treats agent output as an untrusted proposal until independent checks run, and reports what actually happened: passed, failed, unavailable, and unperformed checks are reported separately. *Unknown* is not *passed*. Agent confidence is not evidence. A system that generates reassuring prose while hiding uncertainty is not conforming.

7.5 The topology of friction

TOHSENO aims to make the friction of ordinary creation tend toward zero. It does not aim to make judgment, consent, or consequence disappear. Friction is distributed by the nature of the action:

Local and reversible actions are fast. Creating files, generating a baseline, running tests, launching Simulator, recording an observation, trying an Evolution — as little ceremony as possible.

External and irreversible actions are explicit. Spending money, creating accounts, using signing identities, deploying infrastructure, publishing source, collecting user data, launching an economic object, submitting to the App Store — specific and proximate approval, required every time.

Because the Builder need not be a programmer, approval alone is not enough; approval must be *informed*. Before any external or irreversible action, the Factory must state in plain language what will happen, what it costs, what becomes public or leaves the machine, and what cannot be undone — at the comprehension level of a Builder who cannot read the code. Consent without comprehension is the same failure the protocol forbids in agents, reproduced at the human layer.

TOHSENO tends friction toward zero.
It does not tend responsibility toward zero.

7.6 Ownership principles

Intention first. The Builder begins with the desired application, not with framework selection, account creation, a token model, or a growth strategy. Architecture follows the accepted first form. Infrastructure must earn its existence through application necessity.

Local first. The Shot begins on the Builder's Mac. A selected model provider may receive material when the Builder explicitly chooses it; a chosen dependency may use a network because the application requires it. But no TOHSENO service is the hidden custodian holding the Shot together. Local-first does not mean offline; it means the official service is not the artifact's landlord.

Ejectable from birth. A Shot is ejectable when its owner can build, operate, modify, migrate, or continue it without TOHSENO permission or a hidden factory dependency. Ejection is not a future export feature; it exists at creation. The Builder may remove the Factory entirely and retain the source, history, manifest, tests, and chosen dependencies. Ownership does not require technical mastery of every file. It requires that the Builder can leave.

The Factory is a midwife, not a landlord.

Private provenance. Raw intention can be intimate — unfinished thoughts, personal context, family details, business information. A conforming implementation keeps raw intention and reference material private by default; the tracked repository contains a deliberate, sanitized projection sufficient to explain and continue the application without copying the Builder's thought process into permanent history. The Builder must know which selected provider may receive private material and under whose terms. A provider failure is not permission to silently send the material somewhere else.

Replaceability. A Shot must be able to outlive the planning model, the coding model, the agent harness, the Factory release, the command-line tool, the official website, and the company that helped create it. A later model may continue the Shot; a human programmer may continue it; a different conforming Factory may continue it; the Builder may leave the protocol entirely and continue it as ordinary software. Temporary intelligence cannot become the ontology of the artifact.

8 Apple's Ground

8.1 A commitment, not a first step

TOHSENO builds native iOS applications with SwiftUI and Apple's toolchain. Not web apps that pretend, not cross-platform compromises, not Android. This is not a limitation waiting to be lifted, and it is not a "first profile" politely awaiting successors. TOHSENO is tooling for the Apple ecosystem, period.

The commitment is a claim about where personal software becomes real. Apple built the only consumer computing stack where hardware, operating system, toolchain, and distribution were

designed as one continuous thought — the ground where an ordinary person's idea can travel from a sentence to a native object in their hand with nothing lost in translation. TOHSENO is a bet on that genius: the Mac is the factory, Simulator is the early verification surface, the iPhone is the site of truth, and the App Store is the summit of distribution. The output is a normal native application, never a phone-shaped web preview or a provider-owned workspace.

8.2 The landlord we chose

A protocol whose deepest commitment is *no landlord* must be honest about building on the most deliberate landlord in computing. A device-installed app requires an Apple ID; free provisioning expires; public distribution requires Apple's ongoing consent. TOHSENO does not hide this, and does not consider it a contradiction.

Ejection is from TOHSENO, not from physics. The protocol abolishes *its own* custody over the Shot; it deliberately accepts Apple's custody over the platform, because that custody is inseparable from the qualities that make the iPhone the strongest site of truth available: the device is in the builder's pocket, the toolchain establishes real facts, and the distribution channel reaches real people. TOHSENO removes every landlord it can and names the one it keeps.

The relationship is alignment, not merely dependence. TOHSENO exists to multiply the number of people who create for Apple's platforms, and its future economic layer is designed to express that alignment explicitly (§10.4).

8.3 The creation sequence

```
private intention
  -> bounded, sanitized plan
  -> independently owned baseline
  -> open-ended implementation (AI)
  -> pinned verification (deterministic)
  -> Simulator and device evidence
  -> human Contact
  -> next intention, rest, publication, or another Shot
```

The open-ended steps may use artificial intelligence. The authoritative steps are deterministic. The final application is not claimed to be deterministically generated — model work is inherently variable. What must remain deterministic and inspectable are the boundaries around it: Shot identity, the accepted plan, the composition record, file authority, verification procedures, evidence status, and external approvals.

A Shot may be created and technically verified before it reaches a physical iPhone, but its first creative loop is not complete until a person encounters the application. A conforming Factory therefore makes device use a first-class next action. After creating and verifying a Shot, the system does not bury the Builder in a dashboard. It says, as directly as possible: *open this application*. After Contact, it asks: *what did using this version teach you?* The Factory exists to shorten the distance between those two moments.

8.4 Distribution milestones

```
EVOLVING -> PUBLISHED -> APP_STORE
```

EVOLVING: the application exists primarily in the Builder's local workshop. It may already be complete for its intended private purpose. EVOLVING does not mean unfinished or inferior; it means the Shot is not currently represented through a public channel.

PUBLISHED: the owner makes a verifiable source representation available so others can inspect, build, run, fork, or continue it under its declared terms. Publication is an owner decision, never a requirement for a Shot to be real.

APP_STORE: the owner distributes through Apple's App Store, with the external relationships, signing identities, review, and explicit submission that requires. The Factory may prepare and clarify the path; it may not pretend those authorities have disappeared.

These milestones describe distribution, not creative completion. A private Shot may be deeply successful; an App Store Shot may still need many Evolutions. A public graduation mark may celebrate a threshold crossed. It should never imply that public distribution is the only form of value.

9 Conformance

A protocol is not a brand. It is a shared set of objects, relationships, and constraints that independent implementations can recognize and preserve. Conformance has two parts: requirements an inspector can check, and principles that are judged by conduct. They are listed separately so that neither dilutes the other.

9.1 Verifiable requirements

A conforming implementation:

1. creates one independently operable native iOS repository per Shot;
2. assigns each Shot an opaque, stable identifier at creation, recorded in the manifest and preserved across Evolutions;
3. keeps raw intention and reference material out of the tracked repository by default;
4. discloses which selected intelligence provider receives private input, and routes it nowhere else;
5. runs independent verification on model output before reporting success, and cannot be made to report success without it;
6. reports passed, failed, unavailable, and unperformed checks as distinct states;

7. requires specific, proximate, plain-language approval before every external or irreversible action;
8. records Lived Evidence when the Builder offers it, without requiring public disclosure;
9. leaves behind, at every point in a Shot's life, a repository that builds and runs with the standard Apple toolchain alone — no TOHSENO account, service, token, or binary required.

9.2 Constitutive principles

A conforming implementation also:

1. begins from one coherent human intention, and helps the Builder bound its first form so it is small enough to arrive and complete enough to be encountered;
2. leads the Builder toward Contact rather than treating generation as completion, and asks the canonical question after it;
3. represents meaningful later intentions as Evolutions of the same Shot while semantic continuity remains;
4. permits a Shot to rest without deletion or a declaration of failure;
5. does not measure its own success by the number of applications generated;
6. keeps future identity, network, economic, and publication layers additive and replaceable.

9.3 Recommended properties

A conforming implementation should: provide one fast textual door and one accessible visual door over the same underlying Factory; maintain a visible contact sheet; present one clear next action after creation, and make it Contact whenever possible; separate Machine from Lived Evidence in its interfaces; make data posture and external dependencies explicit; preserve existing Shots through pinned local knowledge rather than silent migration; expose machine-readable interfaces so other tools and agents can inspect and operate a Shot; and keep protocol ceremony behind the interface unless the Builder wants to inspect it.

9.4 The name is not the protocol

The official TOHSENO service cannot make a non-conforming implementation correct by naming it TOHSENO. A fork or independent implementation may realize the protocol without becoming the official service. Trademark, governance, licensing, and conformance are separate questions. The object and its invariants come first.

10 Future Layers

A local Factory producing independently owned Shots creates a larger design space: public identity, signed provenance, shared discovery, structured feedback, collaborative evolution, managed publication, reputation, economic objects, and networks of compatible Factories or verifiers. These systems may be valuable. They are not the Shot, and every one of them is bound by the same constitutional rule: additive, replaceable, and never mandatory for ordinary creation, ownership, use, or continuation.

10.1 Public memory

A public record may preserve signed claims about a Shot's existence, identity, publication, or verification state. Such a record can attest. It cannot become permission to create or continue the Shot. The local artifact remains valid when the public record is unavailable.

10.2 Network services

A network may mirror published artifacts, route feedback, coordinate testing, or maintain shared indexes. A server may coordinate; it may not become the hidden owner of Shot state, and the official service cannot become the definition of the network or a kill switch over locally owned applications. The first decentralization is object-level: *no implementation owns the Shot*. Network decentralization may follow where real use proves it valuable.

10.3 Mobile clients

A mobile TOHSENO application may help a Builder experience Shots, install test versions, sign actions, gather feedback, or view the contact sheet. It may become an important interface. It cannot become the place source fundamentally belongs. The mobile client participates in continuity; it does not possess it.

10.4 Economic objects

An economic object may be linked to a Shot: it may fund development, coordinate participation, reward testing, or align a community around the application. It is not the Shot. No token, asset, payment, stake, or financial relationship may become mandatory for ordinary creation, ownership, use, Evolution, publication, or App Store release.

The protocol-level economic object, `$TOHSENO`, is planned to be paired against `$AAPL` — a deliberate, legible expression of the commitment in §8: TOHSENO's economics aligned with the ecosystem it builds for, rather than extracted from it. Whatever form that takes, the constitutional rule above governs it: an economic layer must serve the application-level object. The application-level object must never be invented to justify the economic layer.

10.5 Managed services

TOHSENO or another provider may offer hosting, builds, publication assistance, backups, collaboration, or infrastructure. These services may be convenient and valuable. Convenience must not be confused with custody: a managed service remains conforming only while the Builder can leave without losing the Shot's ability to exist and continue.

11 Permanence and Versioning

This Genesis paper is intended to be content-addressed and permanently published. Its permanence does not mean TOHSENO will never evolve; it means later evolution cannot silently rewrite what this version said. Future papers may clarify, extend, or supersede this one. They should identify their exact version, reference the prior document, state which definitions or invariants changed, distinguish protocol changes from implementation changes, and preserve the ability of existing Shots to identify the protocol version under which they were created. A later release does not retroactively alter the meaning of an earlier Shot.

The paper should remain smaller than its consequences.

12 Constitutional Invariants

The irreducible commitments of TOHSENO:

1. One coherent intention may be given one bounded, executable attempt. Taking a Shot is the act; the Shot is the stable application lineage created by it.
2. A Shot is a question asked of reality in executable form. Its first form must be small enough to arrive and real enough to answer.
3. The Shot is the application-level object — not the prompt, model, template, build, account, service, token, or distribution channel around it.
4. The first intention creates the Shot. Later grounded intentions create Evolutions of the same Shot. Not every commit is an Evolution; an Evolution answers something learned.
5. Contact, not generation, completes the creative loop. The next intention should emerge from Contact, not automatic expansion.
6. A Shot produces an application and evidence. Machine Evidence and Lived Evidence are distinct and must not be confused. Every readiness claim is bounded by evidence that actually exists.
7. Artificial intelligence may propose; deterministic machinery may prove; the Builder may authorize; reality may answer. A coding agent cannot certify its own work.

8. Reversible local actions are fast. External and irreversible actions require explicit, informed human approval — legible to a Builder who cannot read the code.
9. Raw intention is private by default. Public context is a deliberate projection.
10. A Shot is independently owned and ejectable from birth. Ownership is possession of the repository; the Shot has one Builder of record at a time. The Factory is a midwife, not a landlord.
11. No TOHSENO account, backend, mobile app, token, network, or official server is required to create, own, use, build, or continue a Shot. The Shot must be able to outlive the model, agent, Factory, service, and company that helped create it.
12. A future public record may attest to a Shot; it may not grant it permission to exist. A future network may coordinate around a Shot; it may not become its owner. A future economic object may be linked to a Shot; it is not the Shot.
13. TOHSENO builds for the Apple ecosystem. The Mac is the factory, the iPhone is the site of truth, the App Store is the summit of distribution. This is the protocol's identity, not its first step.
14. A Shot may rest without being declared dead. Most Shots may never become lasting public products; the cost of taking another must remain low. Cheap creation does not remove standards — it moves judgment from speculation to experience.
15. The name TOHSENO does not override conformance. The protocol exists to create more capable creators, not merely more software.

13 Conclusion

The scarce thing in software is no longer code. It is coherent intention carried into contact with reality.

When implementation was expensive, the world filtered intentions by the cost of production. People had to justify an application before they could touch it — to act like founders before they were allowed to behave like creators. That constraint is weakening. A person can now notice something in their life, describe what should exist, give the intention a bounded body, and open it on the device in their hand. They can discover that they were wrong, or almost right, or that the smallest part of the idea was the real one. They can continue it, publish it, let it rest, or take another Shot.

The application is one consequence. The person becoming more capable of creation is another. Over time, the contact sheet becomes evidence of a life spent allowing ideas to meet reality: some Shots become tools, some become public applications, some become companies. Many simply teach the Builder where to aim next.

TOHSENO does not decide which consequence deserves to happen. It does not promise that every idea is good, and it does not ask the Builder to know the ending before beginning. It makes the first honest attempt cheap enough to exist, real enough to answer, and free enough to leave.

TOHSENO makes reality cheap enough to answer.

Give every idea a shot.

Let reality continue it.

Take another one.

Glossary

Builder — the human who supplies the intention, owns the artifact, experiences it, interprets lived evidence, authorizes external actions, and decides what happens next.

Builder of record — the single current owner of a Shot; ownership is possession of the repository.

Coherence — the property by which an intention points toward one application-level object with a durable one-line purpose, one bounded first experience, and explicit non-goals.

Contact — a human encounter with a functioning form of the Shot in a context where its central behavior can become true or false.

Contact sheet — the Builder's collection of Shots: an artifact directory and the visible body of work through which patterns and taste accumulate.

Ejection — the property that permits a Shot to be built, operated, changed, migrated, or continued without TOHSENO permission or a hidden Factory dependency.

Evidence — a grounded result produced through creation or use. *Machine Evidence*: structured results backed by commands, compilers, tests, launches, or hashes that actually executed. *Lived Evidence*: grounded observations produced through human use.

Evolution — a meaningful change to the same Shot made in response to a later intention, grounded in something learned through Contact.

Factory — the deterministic system that protects protocol invariants, establishes identity, validates plans, composes artifacts, bounds agent authority, verifies results, and presents evidence.

First form — the earliest independently operable state of a Shot: bounded enough to arrive, complete enough to produce Contact.

Intelligence Provider — the selected model, agent, or coding environment used to interpret intention and perform open-ended implementation work.

Private provenance — the raw intention, references, and creation context retained privately rather than copied into tracked or public source.

Rest — the legitimate condition in which a Shot has no current intention driving another Evolution but remains whole and available.

Shot — a coherent software intention embodied as a stable, independently owned native iOS application lineage. *Taking a Shot* is the act of giving one coherent intention a bounded, executable attempt.

Taste — judgment developed through repeated distinctions made across Contact with a body of work.

TOHSENO — the protocol and open-source factory for turning coherent intentions into independently owned iOS applications, bringing them into Contact with reality, and preserving the applications and evidence across continuing creation.

END OF GENESIS PAPER

Give every idea a shot. Let reality continue it. Take another one.